Code Explanation

Data Processing Pipeline Code Explanation

This code is a data processing pipeline written in Python using Apache Spark. It reads customer and order data from CSV files, joins the data, and creates or updates a Slowly Changing Dimension (SCD) Type 2 table. It also creates a customer aggregate spend table.

Overview of the Code

The code is organized into several functions that perform different tasks in the data processing pipeline. The main function, `run_batch_processing`, orchestrates the entire pipeline by calling other functions to read data, create or update the SCD2 table, and create the customer aggregate spend table.

Step 1: Reading Customer Data
The `read_customer_data` function reads customer data from a CSV file. It defines a schema for the customer data and uses Spark's `read.csv` method to read the data.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

return (spark.read
        .option("header", "true")
        .schema(customer_schema)
        .csv(source_path)
        .dropDuplicates()
        .filter(col("CustId").isNotNull() &
                col("Name").isNotNull() &
                col("EmailId").isNotNull() &
                col("Region").isNotNull()))
```

Step 2: Reading Order Data
The `read_order_data` function reads order data from a CSV file. It defines a schema for the order data and uses Spark's `read.csv` method to read the data. It also calculates the total amount for each order.

```
order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])

return (spark.read
        .option("header", "true")
        .schema(order_schema)
        .csv(source_path)
        .dropDuplicates()
```

```
        .filter(col("OrderId").isNotNull() &
                col("ItemName").isNotNull() &
                col("PricePerUnit").isNotNull() &
                col("Qty").isNotNull() &
                col("Date").isNotNull() &
                col("CustId").isNotNull())
        .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
)
```

Step 3: Creating or Updating SCD2 Table

The `create_or_update_scd2_table` function creates or updates an SCD2 table by joining customer and order data. It checks if the table exists, and if not, creates it with the required SCD2 fields.

```
# Check if the table exists
table_exists = False
try:
    spark.sql(f"SELECT 1 FROM {catalog}.{schema}.ordersummary LIMIT 1")
    table_exists = True
except:
    pass

# Join customer and order data
joined_df = (customer_df.join(order_df, "CustId")
             .select("CustId", "Name", "EmailId", "Region",
                     "OrderId", "ItemName", "PricePerUnit", "Qty", "Date"))

if not table_exists:
    # First time load - create the table with SCD2 fields
    (joined_df
     .withColumn("IsActive", lit(True))
     .withColumn("StartDate", current_timestamp())
     .withColumn("EndDate", lit(None).cast(TimestampType()))
     .write
     .format("delta")
     .mode("overwrite")
     .saveAsTable(f"{catalog}.{schema}.ordersummary"))
    return
```

Step 4: Implementing SCD2 Logic

The `create_or_update_scd2_table` function implements SCD2 logic by merging the joined data into the target table. It updates existing records and inserts new records.

```
# Perform the merge operation
(target_table.alias("target")
 .merge(
     joined_df.alias("source"),
     join_condition)
 .whenMatchedAnd(update_condition)
 .updateExpr({
     "IsActive": "false",
     "EndDate": "current_timestamp()"
```

```
 })
 .whenNotMatchedInsertExpr({
     "CustId": "source.CustId",
     "Name": "source.Name",
     "EmailId": "source.EmailId",
     "Region": "source.Region",
     "OrderId": "source.OrderId",
     "ItemName": "source.ItemName",
     "PricePerUnit": "source.PricePerUnit",
     "Qty": "source.Qty",
     "Date": "source.Date",
     "IsActive": "true",
     "StartDate": "current_timestamp()",
     "EndDate": "null"
 })
 .execute())
```

Step 5: Creating Customer Aggregate Spend Table

The `create_customer_aggregate_spend` function creates a customer aggregate spend table by aggregating data from the SCD2 table.

```
# Aggregate data from ordersummary
agg_df = (spark.table(f"{catalog}.{schema}.ordersummary")
          .filter(col("IsActive") == True)
          .groupBy("Name", "Date")
          .agg(spark_sum("PricePerUnit" * "Qty").alias("TotalAmount"
)))

# Write to target table
agg_df.write.format("delta").mode("overwrite").saveAsTable(f"{catalo
g}.{schema}.customeraggregatespend")
```

Step 6: Running the Batch Processing Pipeline

The `run_batch_processing` function runs the entire data processing pipeline by calling other functions to read data, create or update the SCD2 table, and create the customer aggregate spend table.

```
# Configuration
catalog = "gen_ai_poc_databrickscoe"
schema = "sdlc_wizard"
customer_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/custo
merdata"
order_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdat
a"

# Read source data
customer_df = read_customer_data(spark, customer_path)
order_df = read_order_data(spark, order_path)

# Create or update SCD2 table
create_or_update_scd2_table(spark, customer_df, order_df, catalog, s
chema)

# Create customer aggregate spend
```

```
create_customer_aggregate_spend(spark, catalog, schema)
```